D

Dumbhanengue ERP

Build et publication par plateforme

Version 1.1.1+3 · 16 août 2026

Dumbhanengue ERP

Table des matières

Prérequis globaux 3
Résumé par plateforme 3
Android 4
- Exigences 4
- Signature de release 4
- Commandes 4

iOS 5
- Exigences 5
- Commandes 5
- Publication sur l'App Store 6

Windows 6
- Exigences 6
- Commandes 6

Web 6
- Exigences 6
- Commandes 7
- Déploiement 7

Linux et macOS (desktop) 7
CI/CD (GitHub Actions) 7
Checklist rapide avant publication 8

> Ce guide couvre comment compiler l'app sur chaque plateforme (Android, iOS, Windows et Web) et les exigences de chacune. Pour la configuration initiale du projet, les variables d'environnement et les conventions, voir le Guide du développeur. Pour la checklist de pré-release, voir la Checklist de production.

Prérequis globaux

- Flutter (canal stable) avec Dart 3.12+ — pubspec.yaml exige sdk: ^3.12.2 ;
- Avant tout build, validez l'environnement :

```
flutter doctor -v
flutter pub get
```

- Version de l'app : version: 1.1.0+2 dans pubspec.yaml (appliquée automatiquement à versionCode/ versionName sur Android et CFBundleVersion sur iOS).

> Mode SaaS vs. local (flavors dev/prod) : les identifiants sont fournis via --dart-define-from-file — env/dev.env (commité, projet dev) et env/prod.env (jamais versionné ; modèle dans env/prod.env.example). FLUTTER_APP_FLAVOR est un define réservé injecté depuis --flavor. Un build release sans --flavor prod lève une erreur au démarrage (main.dart fail-fast) ; de plus, SupabaseBootstrap lève une erreur dans un release NON-prod avec les défauts dev embarqués (fail-fast S2 — ajusté Aug 9 : le déploiement utilise UN SEUL projet Supabase dotklbpqmlnhcdifiukd partagé par dev/prod, donc un release prod avec les identifiants du projet est légitime) ; la configuration Gradle échoue aussi `prodRelease` sans `android/key.properties`. La clé anon est publique par conception — la vraie sécurité vient du RLS ; n'embarquez jamais la clé *service role* :

```
# Développement (debug/hot reload)
flutter run --flavor dev --dart-define-from-file=env/dev.env
# Production (release)
flutter build appbundle --flavor prod --release \
  --dart-define-from-file=env/prod.env
```

Résumé par plateforme

Plateforme	OS requis pour construire	Principales exigences	Commande de release	Artefact généré
Android	Windows, macOS, Linux	JDK 17 + Android SDK + key.properties	flutter build appbundle --flavor prod --release --dart-define-from-file=env/prod.env	app-prod-release.aab

iOS	macOS (obligatoire)	Xcode + CocoaPods + compte Apple Developer	flutter build ios --release	Runner.app (ou .ipa via Xcode)
Windows	Windows 10/11	Visual Studio 2022 (workload C++)	flutter build windows --release	dumbhanengue.exe
Web	Tout OS	Un navigateur (pour les tests)	flutter build web --release	build/web/ (site statique)

Android

Exigences

- JDK 17 (build.gradle.kts utilise JavaVersion.VERSION_17 — la CI utilise Temurin 17) ;
- Android SDK (installé par Android Studio) ; compileSdk, minSdk et targetSdk suivent les défauts Flutter (flutter.compileSdkVersion, etc.) ;
- Pour publier : un keystore de signature (voir ci-dessous) et un compte développeur Google Play Console (frais unique de 25 US$).

Signature de release

La signature de release lit android/key.properties (jamais versionné — voir android/key.properties.example et le Guide du développeur) :

- Avec key.properties présent → signature production (upload keystore + Play App Signing) ;
- Sans le fichier (CI/dev) → le build de release retombe sur le keystore debug (non publiable) ;
- Garde Gradle (Aug 5) : un build prodRelease échoue à la configuration sans key.properties — un AAB de production ne peut jamais être signé debug. Le build CI (ci.yml) utilise donc le flavor dev (com.dumbhanengue.dumbhanengue.dev) purement comme validation de compilation.

Commandes

```
#  Flavors
# dev  → applicationId com.dumbhanengue.dumbhanengue.dev, libellé « Dev »
# prod → applicationId de production com.dumbhanengue.dumbhanengue
# APK dev (distribution directe / test)
flutter build apk --release --flavor dev --dart-define-from-file=env/dev.env
# → build/app/outputs/flutter-apk/app-dev-release.apk
# AAB PROD — le format exigé par Google Play
flutter build appbundle --flavor prod --release \
  --dart-define-from-file=env/prod.env
# → build/app/outputs/bundle/prodRelease/app-prod-release.aab
# AAB PROD avec obfuscation Dart (recommandé avant publication)
flutter build appbundle --flavor prod --release \
  --dart-define-from-file=env/prod.env \
  --obfuscate --split-debug-info=build/symbols
# Tester sur un appareil/émulateur (debug, avec hot reload)
flutter run --flavor dev --dart-define-from-file=env/dev.env
```

> R8/minify sont ACTIVÉS (isMinifyEnabled/isShrinkResources, Aug 5) — le code Dart peut en plus être obfusqué avec --obfuscate --split-debug-info (uploadez les symboles vers Sentry : sentry-cli debug-files upload). Le flavor prod garde l'applicationId immuable com.dumbhanengue.dumbhanengue ; le flavor dev a le suffixe .dev pour que les deux coexistent sur un appareil.

Avertissement de build connu (sentry_flutter + KGP) — suivi upstream

> Aug 8 : Flutter 3.44+ avec AGP 9+ affiche un avertissement en configurant Gradle pour TOUT build Android (dev ou prod) :
>
> ``
> WARNING: Your app uses the following plugins that apply Kotlin Gradle Plugin (KGP): sentry_flutter
> Future versions of Flutter will fail to build if your app uses plugins that apply KGP.
> `
>
> - **Bénin aujourd'hui** — ce n'est qu'un avertissement informatif ; le build se termine normalement (vérifié sur assembleDevRelease). Seules les **futures** versions de Flutter échoueront sur ce point.
> - **Cause :** le android/build.gradle de sentry_flutter applique kotlin-android (KGP) inconditionnellement, et le tooling de Flutter signale ces plugins quand AGP ≥ 9 (ce projet utilise 9.3.1). Il n'existe **aucune** version publiée qui corrige cela (9.26.0, 10.0.0-alpha.3 et le main de sentry-dart appliquent toujours KGP) et aucun flag de suppression dans Flutter 3.44.
> - **Suivi upstream :** **getsentry/sentry-dart#3738** (DART-382) — la migration du plugin vers le Kotlin intégré est visée pour la **v10 stable (~fin août/mi-sept 2026)**, bloquée sur le package jni (dart-lang/native#3415).
> - **Action quand v10 sort :** passez sentry_flutter à ^10.0.0 dans pubspec.yaml` — l'avertissement disparaît de lui-même.

---

iOS

Exigences

- macOS avec Xcode (les builds iOS ne sont possibles que sur macOS) ;
- CocoaPods (ou le gestionnaire de packages natif d'Xcode) — le Podfile est généré automatiquement au premier build ;
- Compte Apple Developer (99 US$/an) avec un certificat de distribution et un provisioning profile ;
- Cible de déploiement minimale : iOS 13.0 (IPHONEOS_DEPLOYMENT_TARGET).

Commandes

```
# Build de release avec signature (exige compte/certificat configurés dans Xcode)
flutter build ios --release
# Build LOCAL sans signature (test, sans compte développeur)
flutter build ios --release --no-codesign
# → build/ios/iphoneos/Runner.app
# Exécuter sur simulateur/appareil (debug)
flutter run
```

Publication sur l'App Store

1. Ouvrez ios/Runner.xcworkspace dans Xcode ;
2. Sélectionnez l'équipe de signature sous *Signing & Capabilities* ;
3. Product → Archive, puis uploadez via App Store Connect (Transporter ou *Distribute App*) ;
4. Configurez l'app dans App Store Connect (métadonnées, confidentialité, captures d'écran) et soumettez à la revue.

> supabase_flutter et flutter_secure_storage fonctionnent sur iOS sans configuration supplémentaire. Pensez à déclarer les autorisations appropriées dans Info.plist si vous ajoutez un jour caméra/localisation.

Windows

Exigences

- Windows 10/11 avec Visual Studio 2022 — installez le workload « Desktop development with C++ », qui inclut les toolchains exigés par Flutter.

Commandes

```
# Build de release (exe natif)
flutter build windows --release
# → build/windows/x64/runner/Release/dumbhanengue.exe
# Exécuter en mode debug avec hot reload
flutter run -d windows
```

L'exécutable généré dépend des DLL du dossier Release — distribuez le dossier entier (ou empaquetez-le avec MSIX via l'outil msix / « Add App Package » de Visual Studio pour le Microsoft Store).

Web

Exigences

- Tout OS (le build s'exécute en Dart pur — pas de toolchain native) ;
- Un navigateur pour les tests (flutter run -d chrome).

Commandes

```
# Build de production (site statique prêt à déployer)
flutter build web --release
# → build/web/
# Mode dev avec hot reload
flutter run -d chrome
# (Optionnel) Build WebAssembly — voir les avertissements de `flutter build web`
flutter build web --release --wasm
```

Déploiement

build/web/ est un site statique : hébergez-le sur n'importe quel serveur/CDN (Firebase Hosting, Vercel, Netlify, GitHub Pages, nginx…). Configurez un repli vers index.html (SPA) si vous utilisez des routes.

> Limitation web : le sqlite3 natif ne s'exécute pas dans le navigateur (dart:ffi n'existe pas). Sur cette plateforme, l'app utilise les datasources en mémoire (mêmes contrats — UI identique), et SaaS (Supabase) est le mode de production recommandé pour le web. Le stockage sécurisé sur web est aussi plus faible (la clé de chiffrement vit dans localStorage).

Linux et macOS (desktop)

Les dossiers linux/ et macos/ n'ont pas été créés dans ce projet pour l'instant. Pour les activer :

```
flutter create --platforms=linux,macos .
```

Puis exécutez flutter build linux --release (exige GTK/clang) et flutter build macos --release (exige macOS + Xcode).

CI/CD (GitHub Actions)

`.github/workflows/ci.yml` (chaque push/PR) :

1. analyze — flutter analyze (zéro erreur/avertissement) ;
2. test — flutter test --coverage + seuil de couverture 45% ;
3. build — flutter build apk --release --flavor dev --dart-define-from-file=env/dev.env (dev uniquement, validation de compilation — signature debug) + artefact APK ;
4. sql-check — scripts/sql_check.sh contre un vrai service Postgres 16 (application du schéma + super admin + RLS par magasin + exécution des RPC atomiques).

`.github/workflows/release.yml` (tag v* — AAB publiable) : exécute analyze + test, puis Fastlane (fastlane/Fastfile, lane deploy) :

1. valide les defines/secrets de production (fail-fast) ;
2. décode le keystore d'upload depuis KEYSTORE_BASE64 et écrit android/key.properties ;
3. construit l'AAB prod (--flavor prod --release --obfuscate --split-debug-info=build/symbols) ;
4. uploade vers le track Internal testing via le compte de service de PLAY_SERVICE_ACCOUNT_JSON.

> Secrets requis : SUPABASE_URL_PROD, SUPABASE_ANON_KEY_PROD, KEYSTORE_BASE64, KEYSTORE_PASSWORD, KEY_ALIAS, PLAY_SERVICE_ACCOUNT_JSON (+ SENTRY_DSN facultatif). Configuration complète : docs/pt/13-build-e-publicacao.md.

Checklist rapide avant publication

- [ ] flutter analyze et flutter test verts ;
- [ ] flutter doctor -v sans problème critique pour la plateforme cible ;
- [ ] Android : Application ID final + key.properties + AAB testé dans Play Console (Internal testing) ;
- [ ] iOS : équipe de signature dans Xcode + archive testée sur TestFlight ;
- [ ] Windows/Web : build de release local validé et artefact distribuable ;
- [ ] SaaS : flavor prod + env/prod.env pointant vers le projet Supabase de production ;
- [ ] Relisez la Checklist de production et le Rapport d'audit.